

République Algérienne Démocratique et Populaire
ministère de l'Enseignement Supérieur et de la Recherche
Scientifique



Binome1 :

Nom: SARNI Hanane

Matricule : 212131014099

Binome2 :

Nom : CHALAL Lydia

Matricule :

Niveau : 212231449416

Master 1 _ INGENIERIE LOGIELLE

Mini-projet multimédia
Pipeline simplifié d'encodeur vidéo MPEG-4

universitaire : 2025/2026

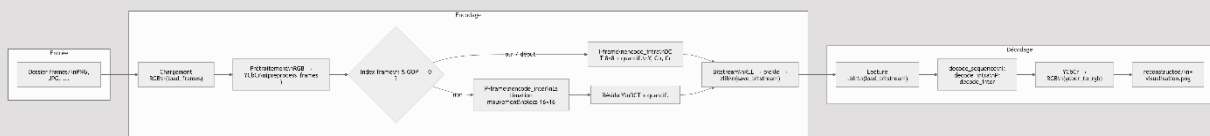
Année

A) Description du pipeline

Vue d'ensemble

Ce projet implémente un **mini encodeur / décodeur vidéo simplifié** en Python : une séquence d'images RGB est traitée image par image, avec une structure de type **GOP** (Group of Pictures) : périodiquement une **image I** (intra, codée seule), puis des **images P** (inter, prédiction temporelle + résidu). Les données sont ensuite **mises en forme, compressées** (RLE + pickle + zlib) et écrites dans un fichier binaire .bin. Le même pipeline relit ce fichier, **décode** la séquence et sauvegarde les images reconstruites, avec des **métriques** (taille, taux de compression approximatif, PSNR) et une **figure de visualisation**.

Schéma du pipeline



Détail de chaque étape

- Chargement des images** (utils.load_frames dans main.py)
Les fichiers image du dossier frames/ sont listés et triés ; chaque image est ouverte en **RGB** et convertie en tableau numpy (valeurs flottantes). Formats pris en charge : PNG, JPG, JPEG, BMP.
- Prétraitement couleur** (preprocessing.preprocess_frames, utils.rgb_to_ycbcr)
Chaque frame RGB est convertie en **YCbCr** (séparation luminance / chrominance), ce qui est classique en compression vidéo : la luminance **Y** porte surtout le détail perçu (tout ce que ton œil voit clairement dans une image) ; **Cb** et **Cr** encodent la couleur (information de couleur appliquée sur la structure (Y)).
- Décision I-frame / P-frame** (main.run_pipeline, paramètre --gop)
 - Si l'indice de la frame vérifie **i % gop == 0** (ou qu'il n'y a pas encore de frame de référence), la frame est traitée comme **I-frame**.
 - Sinon, c'est une **P-frame**, codée par rapport à la reconstruction de la frame précédente.
- Encodage intra (I-frame)** (intra.encode_intra) //////////////////////////////////////
Pour **Y**, **Cb** et **Cr** séparément :
 - Padding** des dimensions à des multiples de 8 (pad_to_block).
 - Découpage en blocs **8×8**, centrage autour de 128.
 - DCT 2D** (matrice orthogonale dct_matrix dans utils.py).
 - Quantification** avec des matrices type JPEG (Q_LUMA pour Y, Q_CHROMA pour Cb/Cr), multipliées par un facteur quality_scale (plus la valeur est grande, plus la compression est forte).

- Coefficients quantifiés stockés en **int16**.
Le résultat est un dictionnaire avec le type "I", la forme, quality_scale, et les trois plans qy, qcb, qcr.

5. Encodage inter (P-frame) (inter.encode_inter)

- Sur le canal **Y** uniquement : **estimation de mouvement** par blocs **16×16** dans une fenêtre de recherche (décalages limités), en minimisant la **SAD** (somme des différences absolues) entre le bloc courant et le bloc de référence dans la **Y** de la frame précédente reconstruite.
- Construction d'une **image prédite** en copiant les blocs déplacés (_build_prediction).
- **Résidu** = Y courante – prédiction ; le résidu est décalé de +128 pour rester dans une plage exploitable, puis compressé comme une image intra avec **la même chaîne DCT + quantification** que la luma (_block_dct_quant avec Q_LUMA).
- La structure encodée contient le type "P", les **vecteurs de mouvement**, qres, etc.
Remarque pour le rapport : en P-frame, la chrominance reconstruite reprend en pratique celle de la référence précédente (le décodeur met à jour surtout Y), ce qui est une **simplification pédagogique** par rapport à un codec réel.

6. Entropie et fichier binaire (entropy.save_bitstream)

- Les tableaux de coefficients (qy, qcb, qcr ou qres) sont parcourus en **1D** et encodés en **RLE** (couples valeur / répétition).
- L'ensemble des frames + **métadonnées** (taille de GOP, noms des fichiers, quality_scale) est sérialisé avec **pickle**, puis compressé avec **zlib** niveau 9, et écrit dans **output.bin**.

7. Décodage (decoder.decode_sequence, intra.decode_intra, inter.decode_inter)

- Lecture du bitstream (load_bitstream).
- Pour chaque frame : si **I**, **inverse quantification + IDCT** par bloc 8×8 sur Y, Cb, Cr, **unpad**, clipping ; si **P**, reconstruction de **Y** = prédiction + résidu décodé ; chroma héritée de la frame de référence selon la logique du code.
- Conversion **YCbCr** → **RGB** (utils.ycbcr_to_rgb) pour l'affichage et la sauvegarde.

8. Sorties et évaluation (main.py)

- Images dans **reconstructed/** (mêmes noms que l'entrée).
- **PSNR** moyen entre RGB original et RGB reconstruit (utils.psnr).
- **Taux de compression approximatif** : taille totale des pixels originaux / taille du fichier compressé (approximation pédagogique).
- Figure **visualisation.png** : original, plans Y/Cb/Cr, reconstruction, carte d'erreur.

9. Lancement (cf. README.md)

Exemple :

python main.py --frames_dir frames --output_bin output.bin --gop 5 --quality_scale 1.0
avec typiquement **10 à 30 images** dans frames/ (votre README
mentionne **20 images** pour votre binôme/groupe).

b) Choix de conception et justification

Taille des blocs

Le projet utilise **deux tailles de bloc différentes**, pour deux rôles différents.

Blocs 8×8 pour la DCT / la quantification (intra.py, utils.pad_to_block avec block_size = 8)

La transformée en cosinus discrète (DCT) est appliquée sur des blocs **8×8**, ce qui correspond au **choix historique des standards de type JPEG / MPEG** : un compromis entre coût de calcul, localité spatiale des fréquences et efficacité de la quantification. Un bloc plus petit capture moins de corrélation spatiale ; un bloc plus grand augmente la complexité (matrices plus grandes, plus de coefficients à coder) et rend la prédiction spectrale moins adaptée aux variations locales. Les **tables de quantification** fournies (Q_LUMA, Q_CHROMA) sont elles aussi définies pour **8×8**, ce qui rend ce choix cohérent avec une chaîne « style JPEG » simplifiée.

Blocs 16×16 pour l'estimation de

mouvement (inter.py, _motion_estimation avec block_size = 16)

Le mouvement est estimé sur des macro-blocs **16×16** sur le canal **Y**. C'est un compromis classique en codage vidéo : des blocs **8×8** donnent des vecteurs plus fins mais plus nombreux (plus de données côté mouvement, plus de risque de sur-segmentation du bruit) ; des blocs **32×32** ou plus réduisent le coût en vecteurs mais lissent le champ de mouvement et peuvent mal représenter les petits détails. **16×16** est une taille courante (proche des macro-blocs MPEG/H.26x) et reste **raisonnable en temps de calcul** pour un TP : chaque bloc ne nécessite qu'un seul vecteur (dy, dx) au lieu de quatre vecteurs 8×8 pour la même surface.

Fenêtre de recherche du mouvement

La recherche exhaustive utilise search_range = 4 dans inter.py, soit des décalages **dy et dx de -4 à +4** (fenêtre **9×9** positions par bloc, en excluant les positions qui sortent de l'image).

Justifications:

1. **Complexité** : pour chaque bloc 16×16, le nombre de candidats est borné par $(2R + 1)^2$ avec $R = 4$, soit au plus **81** comparaisons par bloc (moins si le bloc est près du bord). Augmenter R améliore la prédiction sur des mouvements rapides mais augmente **quadratiquement** le temps de calcul, ce qui pénalise une séquence de nombreuses images sur une machine étudiante.
2. **Hypothèse de continuité temporelle** : entre deux images successives d'une vidéo à cadence « normale », le déplacement d'un objet entre deux frames est souvent **modéré**. Une fenêtre ± 4 pixels sur une grille 16×16 reste compatible avec des **petits mouvements** de caméra ou d'objets, objectif raisonnable pour un encodeur pédagogique.

3. **Cohérence avec la résolution** : si les images sont relativement petites ou peu mouvantes, une grande fenêtre n'apporte pas toujours de gain PSNR proportionnel au coût. La valeur **4** est donc un **réglage par défaut** simple, facile à documenter et à faire varier dans une section « perspectives » ou « expériences » du rapport (par ex. comparer PSNR / temps pour `search_range = 2, 4, 8`).

Codage entropique : quel algorithme, et pourquoi ?

Dans `entropy.py`, l'étape « entropique » n'est pas un **codage entropique au sens MPEG strict** (pas de CABAC, pas de Huffman JPEG bit-à-bit ici), mais une **chaîne en deux niveaux** :

1. **RLE (Run-Length Encoding)** sur le flux 1D des coefficients quantifiés (`rle_encode_1d / rle_decode_1d`) : les tableaux `qy`, `qcb`, `qcr` ou `qres` contiennent beaucoup de **zéros** après quantification ; en parcourant le bloc dans un ordre fixe (`flatten()`), des **runs** de valeurs identiques (souvent 0) apparaissent. La RLE remplace des séquences répétées par des paires (**valeur, nombre de répétitions**), ce qui **réduit la redondance** sans implémenter un modèle probabiliste complexe.
2. **pickle + zlib** : la structure Python (métadonnées + frames packées) est sérialisée puis compressée avec **DEFLATE** (zlib, niveau 9). DEFLATE combine **LZ77** (dictionnaire de chaînes répétées) et **Huffman** sur les symboles produits ; c'est donc un **codage entropique et de redondance** générique au niveau **fichier entier**, appliqué après la RLE.

Pourquoi ce choix dans le projet / TP :

- **Simplicité et robustesse** : pas besoin d'implémenter des tables Huffman, un parseur de bitstream binaire fragile, ou un arithmétique coding ; le code reste court et lisible.
- **Gain réel** : la RLE exploite la **structure des coefficients DCT quantifiés** ; zlib compresses encore les motifs répétitifs dans le flux pickle (entêtes, structures répétées entre frames).
- **Limite honnête à mentionner** : dans un codec industriel, l'entropie est **par symbole** (contexte, CABAC, etc.) et le bitstream est **standardisé** ; et la l'objectif est **pédagogique** (pipeline complet encode → fichier → decode) plutôt que l'interopérabilité ou l'optimalité bit-rate stricte.

c) Analyse expérimentale

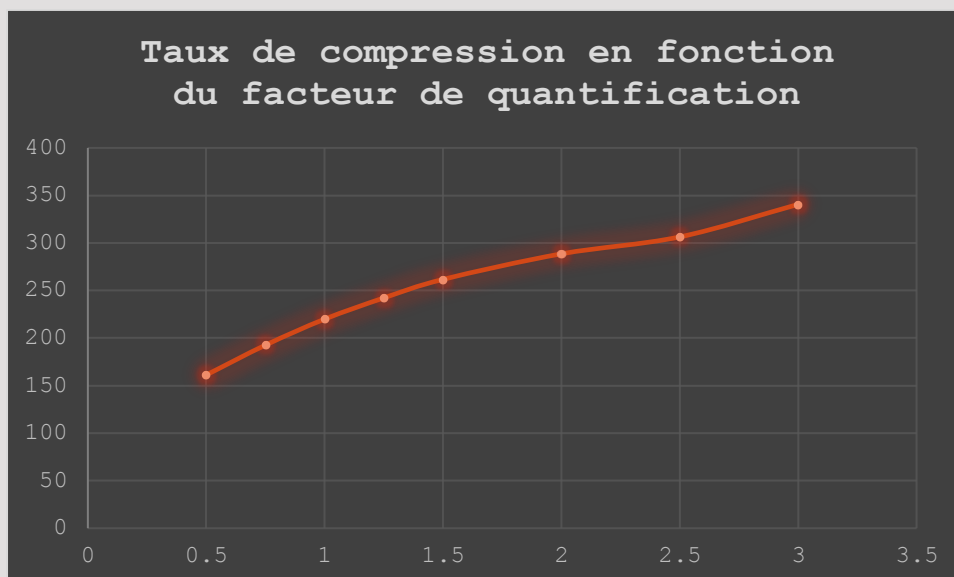
Protocole

- **Séquence** : 20 images RGB **160×120** générées automatiquement (dégradé + motif sinusoïdal + carré rouge dont la position varie légèrement d'une frame à l'autre), pour avoir à la fois de la **texture** et un peu de **mouvement** exploitable par les P-frames.
- **Taille brute** : somme des $H \times W \times 3$ octets de toutes les frames (comme dans `main.py`).
- **Taille compressée** : octets du fichier `.bin` après RLE, pickle et zlib (`save_bitstream`).
- **Taux de compression** : $\text{taille brute} \div \text{taille du fichier compressé}$ (plus la valeur est **grande**, plus le fichier est **petit** par rapport à l'RAW).
- **Courbe 1** : `quality_scale` ∈ {0.5, 0.75, 1.0, 1.25, 1.5, 2.0, 2.5, 3.0}, **GOP = 5** fixe.

- **Courbe 2** : $\text{GOP} \in \{1, 2, 3, 4, 5, 6, 8, 10, 15, 20\}$, **quality_scale = 1.0** fixe.

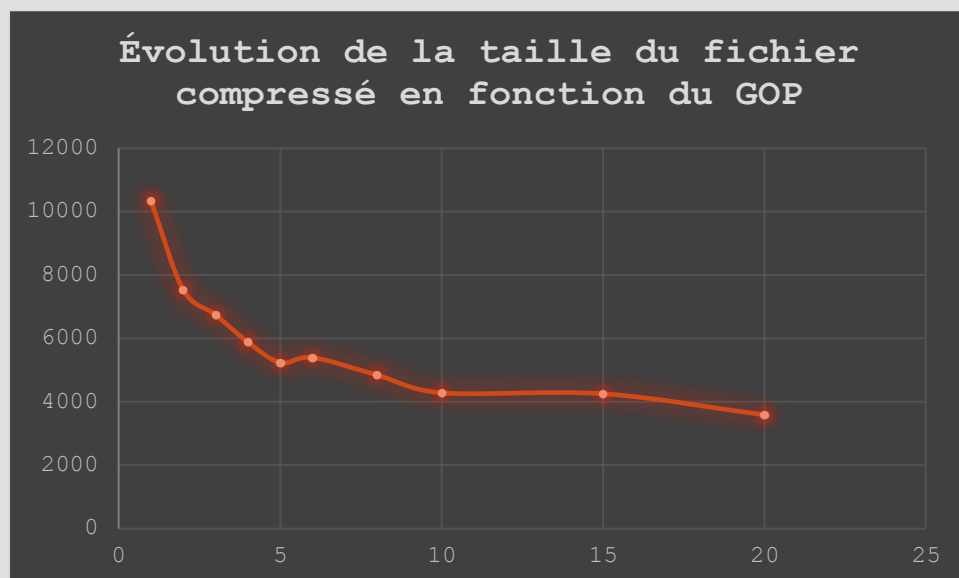
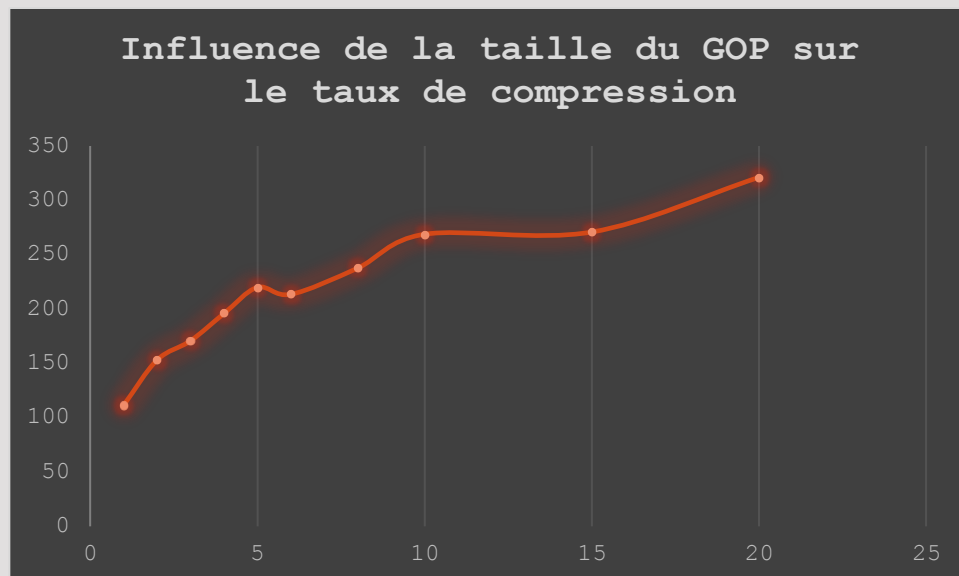
Courbe 1 — Taux de compression en fonction du facteur de quantification

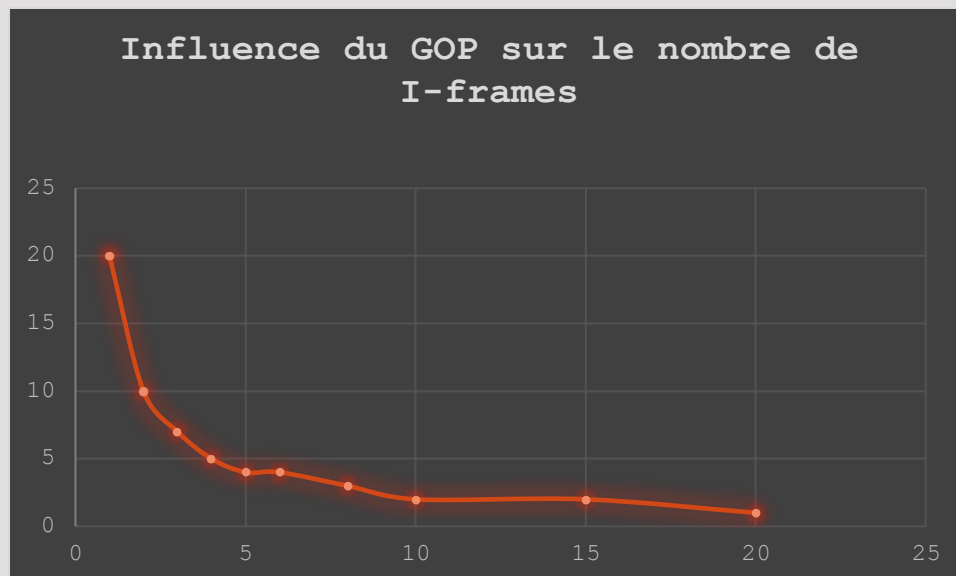
quality_scale	Taille .bin (octets)	Taux de compression	PSNR moyen (dB)
0.5	7146	161.2	26.46
0.75	5982	192.6	25.86
1.0	5238	219.9	25.36
1.25	4757	242.2	24.96
1.5	4402	261.7	24.79
2.0	3992	288.6	24.69
2.5	3760	306.4	24.41
3.0	3382	340.6	24.25



Courbe 2 — Influence de la taille du GOP sur la compression

GOP	Taille .bin (octets)	Taux de compression	Nb. I-frames (sur 20)
1	10345	111.4	20
2	7531	153.0	10
3	6748	170.7	7
4	5884	195.8	5
5	5238	219.9	4
6	5396	213.5	4
8	4848	237.6	3
10	4285	268.8	2
15	4254	270.8	2
20	3588	321.1	1





(Remarque : pour $GOP = 6$ la courbe peut localement **redescendre** par rapport à $GOP = 5$, car le nombre d'I-frames peut rester identique alors que la **répartition** des P-frames change, et la couche zlib sur le tout ne garantit pas une monotone stricte octet par octet.)

Interprétation des résultats (analyse) :

Lorsque **quality_scale augmente**, les pas de quantification deviennent plus grands ($q = \max(1, Q \times \text{quality_scale})$ dans `intra.py`), donc plus de coefficients DCT sont annulés ou simplifiés : le flux après quantification est plus **parcimonieux**, la RLE compresse mieux, et le fichier `.bin` **diminue** ; on observe bien une **hausse du taux de compression** de 161 à environ 341 sur notre séquence, au prix d'une **baisse du PSNR** (de 26.5 dB à 24.2 dB), ce qui traduit le **compromis débit-distorsion** classique de la quantification.

Lorsque le **GOP s'élargit**, le nombre d'**I-frames** diminue (une seule I-frame pour $GOP = 20$ sur 20 images), alors que les **P-frames** codent surtout un résidu après prédiction : pour une scène avec mouvement modéré, le coût cumulé des I-frames domine souvent la taille du bitstream, d'où une **amélioration nette du taux de compression** quand on passe de $GOP = 1$ (toutes intra, 10345 octets) à $GOP = 20$ (3588 octets).

Entre valeurs intermédiaires de GOP, la taille du fichier ne suit pas toujours une courbe strictement monotone, car elle dépend aussi du **contenu** (certains indices deviennent des I-frames selon i % GOP) et de l'efficacité variable de **zlib** sur le flux pickle ; l'**tendance globale** reste néanmoins claire : **moins d'I-frames ⇒ meilleure compression**, au détriment d'une propagation d'erreurs plus longue en cas de décodage partiel et d'une sensibilité accrue aux changements de plan (non modélisés ici).

Tu peux supprimer `run_experiments.py` après ton rapport si tu ne veux pas le rendre avec le projet ; sinon garde-le pour **refaire les courbes** avec ton dossier `frames/` réel (il suffit de remplacer la génération temporaire par `load_frames("frames")`). Si tu veux, je peux adapter le script pour qu'il prenne `--frames_dir` en argument sans images synthétiques.